

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ПРАКТИЧЕСКАЯ РАБОТА 1

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Знакомство с Docker

Обучающийся Пухарев Сергей Валерьевич

Факультет Факультет прикладной информатики

Группа К3320

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Пухарев С.В.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Аминов Н.С.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ВЕРИФИКАЦИЯ ОКРУЖЕНИЯ И АРХИТЕКТУРЫ	4
1.1 Проверка установки Docker	4
1.2 Состояние локального хранилища	4
1.3 Артефакт окружения	5
2 ПЕРВЫЕ ШАГИ: «ИГРУШЕЧНЫЕ» ОБРАЗЫ	6
2.1 hello world	6
2.2 Минимальный образ	6
2.3 Сравнение поведения двух контейнеров	7
3 ЖИЗНЕННЫЙ ЦИКЛ КОНТЕЙНЕРА	8
3.1 Отслеживание всех состояний контейнера	8
4 ДИАГНОСТИКА РЕАЛЬНОГО СЕРВИСА	9
4.1 Запуск с ошибкой	9
ЗАКЛЮЧЕНИЕ	10

ВВЕДЕНИЕ

Цель работы – ознакомление с базовыми командами Docker-а, созданием контейнеров и командами для диагностики.

1 ВЕРИФИКАЦИЯ ОКРУЖЕНИЯ И АРХИТЕКТУРЫ

Таблица 1 — Мои значения для лабораторной

Номер ИСУ	467203
Персональный порт	5433
Персональный пре-фикс	pukharev

1.1 Проверка установки Docker

На рисунке 1 представлен вывод версии клиента и сервера Docker.

```
seraypuh@Sergejs-MacBook-Air ~ % docker version --format "Client: {{.Client.Version}}, Server: {{.Server.Version}}"
Client: 28.3.2, Server: 28.3.2
seraypuh@Sergejs-MacBook-Air ~ % docker info --format "{{.OSType}}/{{.Architecture}}"
linux/x86_64
seraypuh@Sergejs-MacBook-Air ~ %
```

Рисунок 1 — Версия докера

Версия и клиента, и сервера – 28.3.2

1.2 Состояние локального хранилища

На рисунке 2 представлено текущее состояние докера.

```
seraypuh@Sergejs-MacBook-Air ~ % docker system df --format "table {{.Type}}\t{{.TotalCount}}\t{{.Active}}\t{{.Size}}"
TABLE      TOTAL      ACTIVE     SIZE
Images     15         8          12.53GB
Containers  8          0          2.871MB
Local Volumes  6         4          236MB
Build Cache 93         0          1.571GB
seraypuh@Sergejs-MacBook-Air ~ %
```

Рисунок 2 — Состояние

До начала выполнения работы хранится 15 образов и 0 активных контейнеров.

1.3 Артефакт окружения

На рисунке 3 представлен вывод команды для получения артефакта окружения.



```
seraypuh@Sergejs-MacBook-Air ~ % echo "Student: $(whoami)@$(hostname)" && \
echo "Date: $(date '+%Y-%m-%d %H:%M:%S')" && \
docker version --format "Docker {{.Server.Version}} on {{.Server.Os}}/{{.Server.Architecture}}"

Student: seraypuh@Sergejs-MacBook-Air.local
Date: 2026-09-24 10:58:32
Docker 28.3.2 on linux/
template: version:1:54: executing "version" at <.Server.Architecture>: can't evaluate field Architecture in type *types.Version
seraypuh@Sergejs-MacBook-Air ~ %
```

Рисунок 3 — Артефакт окружения

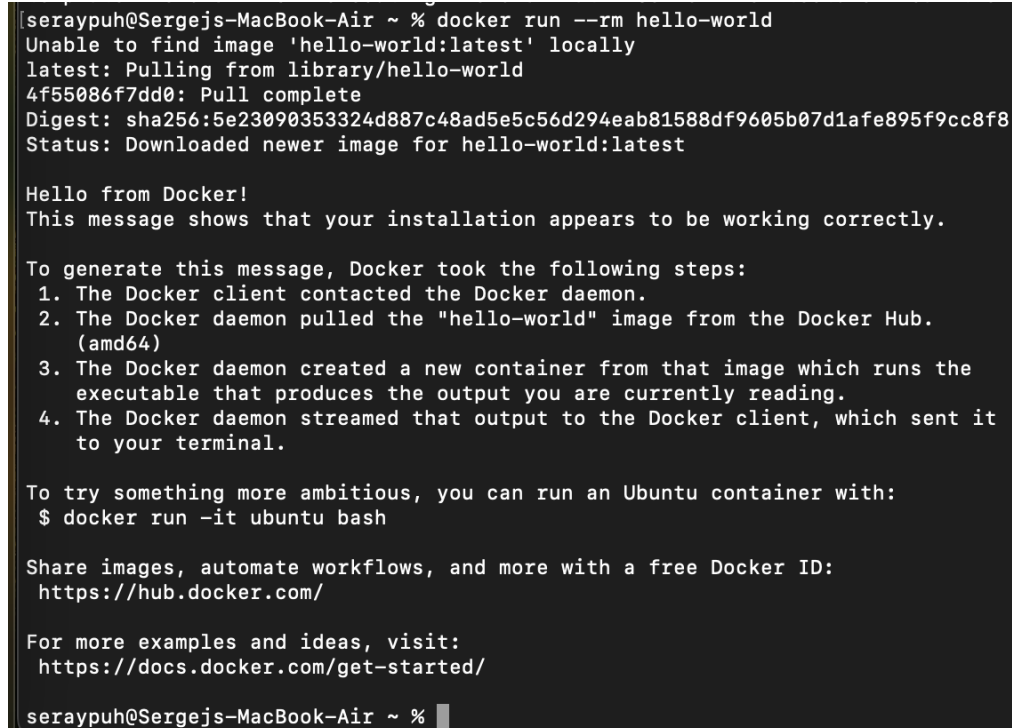
При запуске `docker system prune` без флагов будут утеряны все остановленные контейнеры, все сети без подключенных контейнеров, образы без тега `none` и на которые не ссылается ни один контейнер (висячие образы), и кеш сборки.

Команда `docker version` может выдать ошибку, если докер не запущен, если пользователь не имеет прав для сокета `/var/run/docker.sock`, если версии докер клиента и докер-демона несовместимы.

2 ПЕРВЫЕ ШАГИ: «ИГРУШЕЧНЫЕ» ОБРАЗЫ

2.1 hello world

На рисунке 4 представлен запуск контейнера из образа hello world.

A screenshot of a terminal window on a MacBook Air. The prompt is 'seraypuh@Sergejs-MacBook-Air ~ %'. The user enters 'docker run --rm hello-world'. The output shows that Docker cannot find the image locally, so it pulls it from the Docker Hub library. It displays the image ID '4f55086f7dd0', the pull status 'Pull complete', and the digest 'sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8'. The status is 'Downloaded newer image for hello-world:latest'. Then, the container runs and outputs 'Hello from Docker!' followed by a message confirming the installation is working. It lists the steps Docker took: 1. Docker client contacted the daemon. 2. Docker daemon pulled the 'hello-world' image from Docker Hub (amd64). 3. Docker daemon created a new container from the image. 4. Docker daemon streamed the output to the client. It then suggests running an Ubuntu container with '\$ docker run -it ubuntu bash'. It provides a link to Docker Hub for sharing images and a link to Docker docs for more examples. The prompt returns to 'seraypuh@Sergejs-MacBook-Air ~ %'.

```
seraypuh@Sergejs-MacBook-Air ~ % docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

seraypuh@Sergejs-MacBook-Air ~ %
```

Рисунок 4 — hello world

Образ загружается из [docker.io/library/hello-world:latest](https://hub.docker.com/library/hello-world).

Исходя из вывода, внутри контейнера происходит запуск исполняемого файла, который выводит сообщение.

2.2 Минимальный образ

На рисунке 5 представлен контейнер с минимальным образом linux.

```

seraypuh@Sergejs-MacBook-Air ~ % docker run --rm alpine:3.18 cat /etc/os-release

Unable to find image 'alpine:3.18' locally
3.18: Pulling from library/alpine
44cf07d57ee4: Pull complete
Digest: sha256:de0eb0b3f2a47ba1eb89389859a9bd88b28e82f5826b6969ad604979713c2d4f
Status: Downloaded newer image for alpine:3.18
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.18.12
PRETTY_NAME="Alpine Linux v3.18"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
seraypuh@Sergejs-MacBook-Air ~ % docker images alpine:3.18 --format "{{.Size}}"
11.5MB
seraypuh@Sergejs-MacBook-Air ~ % █

```

Рисунок 5 — Минимальный образ

PRETTY_NAME = "Alpine Linux v3.18"

Размер образа: 11.5 МБ

2.3 Сравнение поведения двух контейнеров

На рисунке 6 представлено создание двух контейнеров.

```

seraypuh@Sergejs-MacBook-Air ~ % docker run -d --name pukharev-alpine-1 alpine:3.18 sleep 300
174080bf8fc2af5af37100f22b171e82ea1d022733d5fc1f1d2c499f6e34cb7d
seraypuh@Sergejs-MacBook-Air ~ % docker run -d --name pukharev-alpine-2 alpine:3.18 sh -c "echo 'Hello from $(hostname)' && sleep 300"
4d99f964d7338da333f89674e9db07af7ba12e37c0b5d9054efa12baaf08efa7
seraypuh@Sergejs-MacBook-Air ~ % docker ps --filter "name=pukharev-alpine"

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
4d99f964d733	alpine:3.18	"sh -c 'echo 'Hello _'"	29 seconds ago	Up 29 seconds		pukharev-alpine-2
174080bf8fc2	alpine:3.18	"sleep 300"	51 seconds ago	Up 50 seconds		pukharev-alpine-1

Рисунок 6 — Два контейнера

Контейнер со sleep300 не завершился сразу, потому что контейнер живет сколько, сколько живёт его процесс с PID 1. В нашем случае PID 1 это sleep на 5 минут, поэтому контейнер не завершается сразу.

Если убрать `--rm`, то контейнер не самоустранился при завершении, а будет висеть в статусе `Exited(0)`. Удалить их можно либо через `docker container prune`, но лучше найти их через `docker ps -a` и удалять по id через `docker rm`.

3 ЖИЗНЕННЫЙ ЦИКЛ КОНТЕЙНЕРА

3.1 Отслеживание всех состояний контейнера

На рисунке 7 представлено отслеживание состояний контейнера.

```
seraypuh@Sergejs-MacBook-Air ~ % docker create --name pukharev-nginx nginx:alpine
605b5ab30563450dbac870731d4871c08b396634290ac2aa333ddcf6bde5b1d
seraypuh@Sergejs-MacBook-Air ~ % docker ps -a --filter "name=pukharev-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS    CREATED AT
pukharev-nginx Created    2026-09-24 12:23:46 +0300 MSK
seraypuh@Sergejs-MacBook-Air ~ % docker start pukharev-nginx
pukharev-nginx
seraypuh@Sergejs-MacBook-Air ~ % docker ps -a --filter "name=pukharev-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS    CREATED AT
pukharev-nginx Up 4 seconds 2026-09-24 12:23:46 +0300 MSK
seraypuh@Sergejs-MacBook-Air ~ % docker stop pukharev-nginx
pukharev-nginx
seraypuh@Sergejs-MacBook-Air ~ % docker ps -a --filter "name=pukharev-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS    CREATED AT
pukharev-nginx Exited (0) 2 seconds ago 2026-09-24 12:23:46 +0300 MSK
seraypuh@Sergejs-MacBook-Air ~ % docker restart pukharev-nginx
pukharev-nginx
seraypuh@Sergejs-MacBook-Air ~ % docker ps -a --filter "name=pukharev-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS    CREATED AT
pukharev-nginx Up 5 seconds 2026-09-24 12:23:46 +0300 MSK
seraypuh@Sergejs-MacBook-Air ~ % docker kill pukharev-nginx
pukharev-nginx
seraypuh@Sergejs-MacBook-Air ~ % docker ps -a --filter "name=pukharev-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS    CREATED AT
pukharev-nginx Exited (137) 6 seconds ago 2026-09-24 12:23:46 +0300 MSK
seraypuh@Sergejs-MacBook-Air ~ % docker rm pukharev-nginx
pukharev-nginx
seraypuh@Sergejs-MacBook-Air ~ % docker ps -a --filter "name=pukharev-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS    CREATED AT
seraypuh@Sergejs-MacBook-Air ~ % █
```

Рисунок 7 — Состояния

Таблица 2 — Таблица состояний

Команда	Состояние
create	Created
start	Up
stop	Exited(0)
restart	Up
kill	Exited(137)
rm	—

По умолчанию docker stop отправляет сигнал SIGTERM – мягкое завершение процесса.

А docker kill отправляет сигнал SIGKILL, который невозможно предотвратить и который завершает процесс принудительно, и он может произойти например во время транзакции.

Если написать docker rm для работающего контейнера, то докер не даст это сделать, так как предусмотрена защита.

А docker rm -f удаляет принудительно, сигналом SIGKILL.

4 ДИАГНОСТИКА РЕАЛЬНОГО СЕРВИСА

4.1 Запуск с ошибкой

На рисунке 8 представлен запуск с ошибкой.

```
seraypuh@Sergejs-MacBook-Air ~ % docker run -d --name pukharev-pg-broken -p 5433:5432 postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
170d9ea1cd85: Pull complete
75a4dff090e8: Pull complete
afe9a40fb214: Pull complete
c95afe7c234f: Pull complete
ca5e97c9cd7d: Pull complete
53fa82bdba52: Pull complete
08a5cd487deb: Pull complete
7b9c6b8e5e6e: Pull complete
eb4d92aee0a: Pull complete
b65147f15943: Pull complete
0d39881c83f7: Pull complete
bc176b99a33a: Pull complete
6b37362b3da7: Pull complete
0d791b2b75ae: Pull complete
Digest: sha256:dfbbb0ad8cab91d41e99123644e37a9b25d0413426d4fb3154e4d668e5d35246
Status: Downloaded newer image for postgres:15
c98f4dd21f34die2ad67c22c4b7914ce03f739c1d4e166a1340110a95a3c7b92
seraypuh@Sergejs-MacBook-Air ~ % docker ps -a --filter "name=pukharev-pg"
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
c98f4dd21f34   postgres:15 "docker-entrypoint.s..." 26 seconds ago Exited (1) 23 seconds ago           pukharev-pg-broken
seraypuh@Sergejs-MacBook-Air ~ % docker logs --timestamps pukharev-pg-broken 2>&1 | tail -20
2026-09-24T10:26:39.204331789Z Error: Database is uninitialized and superuser password is not specified.
2026-09-24T10:26:39.204384439Z You must specify POSTGRES_PASSWORD to a non-empty value for the
2026-09-24T10:26:39.204391359Z superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".
2026-09-24T10:26:39.204394865Z
2026-09-24T10:26:39.204397654Z You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all
2026-09-24T10:26:39.204400656Z connections without a password. This is *not* recommended.
2026-09-24T10:26:39.204403350Z
2026-09-24T10:26:39.204405982Z See PostgreSQL documentation about "trust":
2026-09-24T10:26:39.204409004Z https://www.postgresql.org/docs/current/auth-trust.html
seraypuh@Sergejs-MacBook-Air ~ % docker inspect pukharev-pg-broken --format '{{.State.ExitCode}}'
1
seraypuh@Sergejs-MacBook-Air ~ %
```

Рисунок 8 — Запуск с ошибкой

На рисунке 9 представлен запуск без ошибки.

```
seraypuh@Sergejs-MacBook-Air ~ % docker run -d --name pukharev-pg-fixed -e POSTGRES_PASSWORD=Pass_5433 -e POSTGRES_USER=user_467203 -p 5433:5432 postgres:15
6d4b0f9661e31b669fb1ac4c38b3bae9e679dc92900700f2daefde2c586db558
seraypuh@Sergejs-MacBook-Air ~ % docker exec pukharev-pg-fixed -U user_467203 -c "SELECT version();"
OCI runtime exec failed: exec failed: unable to start container process: exec: "-U": executable file not found in $PATH: unknown
seraypuh@Sergejs-MacBook-Air ~ % docker exec pukharev-pg-fixed psql -U user_467203 -c "SELECT version();"
version
-----
 PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)
seraypuh@Sergejs-MacBook-Air ~ %
```

Рисунок 9 — Запуск без ошибки

На рисунке 10 представлен финальный артефакт работы – работающий контейнер с postgres.

```
seraypuh@Sergejs-MacBook-Air ~ % echo "=== Практика №1: $(date +%Y-%m-%d %H:%M:%S) ===" && \
echo "Студент: $(whoami)@$(hostname)" && \
docker ps --filter "name=pukharev-pg-fixed" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

=== Практика №1: 2026-09-24 13:41:36 ===
Студент: seraypuh@Sergejs-MacBook-Air.local
NAMES          STATUS        PORTS
pukharev-pg-fixed Up 3 minutes  0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
seraypuh@Sergejs-MacBook-Air ~ %
```

Рисунок 10 — Финальный артефакт

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной лабораторной работы были получены навыки создания, удаления и диагностики самых простых контейнеров в Docker.